Day07 出行目的识别

> 课程：交通时空大数据分析与可视化系统（Python 版） | 第 7 天（8 课时）
> 主题：从驻留段中识别出行目的（居住/购物/旅游/其他），结合时间规则和空间匹配

一、本日目标

1. 理解出行目的识别的业务背景和规则设计；
2. 掌握驻留段识别（连续驻留点合并为驻留段）；
3. 掌握时间规则判定（居住>4h、购物9-21点、旅游9-17点）；
4. 掌握出行目的静态数据读取（GBK编码）和预处理；
5. 掌握空间匹配（驻留段与目的点<1km取最近）；
6. 完整实现出行目的模块（purpose.py）；
7. 分析出行目的结果并可视化；
8. 参数调优与边界情况处理。

二、知识要点

- 出行目的识别基于驻留段（连续驻留点合并），不是单个点；
- 时间规则：居住（持续>4h且0-6点起/20-24点止）、购物（9点后起21点前止）、旅游（9点后起17点前止）；
- 空间匹配：驻留段中心与出行目的静态数据<1km取最近；
- 静态数据文件为GBK编码，读取时需指定 encoding="gbk"；
- 最终输出：purpose_result.csv（驻留段信息+出行目的）。

课时1：出行目的识别总览与业务规则

1.1 课时目标

理解出行目的识别的业务背景，掌握四类出行目的的判定规则。

1.2 知识点深度讲解

1.2.1 什么是出行目的

出行目的指用户出行（离开常住地）后到达某个驻留区域的目的。本项目识别四类出行目的：
1. **居住**：用户在该区域长时间停留，且时间覆盖夜间（ likely 是家或酒店）；
2. **购物**：用户在该区域停留，且时间在商场营业时间内（ likely 是商场/超市）；
3. **旅游**：用户在该区域停留，且时间在白天（ likely 是景点/公园）；
4. **其他**：不满足以上三类的驻留段。

1.2.2 为什么需要出行目的识别

出行目的识别是交通时空大数据分析的重要应用：
- **商业分析**：哪些商场人流量大、用户停留时间长；
- **旅游分析**：哪些景点受欢迎、游客来源地；
- **城市规划**：居住区域、商业区域、旅游区域的分布；
- **精准营销**：根据用户出行目的推送相关广告。

1.2.3 判定规则总览

出行目的判定分两步：
1. **时间规则**：根据驻留段的开始时间、结束时间、持续时间初步判定目的；
2. **空间匹配**：驻留段中心与出行目的静态数据（商场/景点位置）匹配，<1km取最近。

时间规则：

目的	持续时间	开始时间	结束时间
居住	>4小时	0-6点 或 20-24点	0-6点 或 20-24点
购物	无特殊要求	9点后	21点前
旅游	无特殊要求	9点后	17点前
其他	不满足以上	-	-

> 居住的判定最严格：持续时间>4小时，且开始或结束时间在夜间（0-6点或20-24点）。这符合"在家过夜"的业务逻辑。

1.2.4 时间规则的优先级

当一个驻留段同时满足多个时间规则时，优先级为：
1. 居住（最严格，优先判定）；
2. 购物；
3. 旅游；
4. 其他。

> 为什么居住优先级最高？因为居住的判定条件最严格（持续>4h且夜间），如果满足，基本可以确定是居住。购物和旅游的条件较宽松，可能重叠。

1.2.5 空间匹配的作用

时间规则只能初步判定目的，空间匹配可以验证和修正：
- 如果时间规则判定为"购物"，且驻留段附近1km内有商场 → 确认购物；
- 如果时间规则判定为"旅游"，且驻留段附近1km内有景点 → 确认旅游；
- 如果时间规则判定为"居住"，不需要空间匹配（家的位置不在静态数据中）；
- 如果空间匹配不到任何目的点 → 保持时间规则的判定或改为"其他"。

1.3 项目实现步骤

1. 读取阶段3结果（驻留/出行标签）；
2. 识别驻留段（连续驻留点合并）；
3. 对每个驻留段应用时间规则；
4. 读取出行目的静态数据（GBK编码）；
5. 空间匹配（<1km取最近）；
6. 输出结果。

1.4 完整代码（出行目的规则演示）

```
# ============================================================
# 课时1：出行目的识别总览与业务规则
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统\09.交通时空大数据分析系统\B-项目数据"

# 读取阶段3结果
df = pd.read_csv(os.path.join(RESULT_DIR, "stage3_label.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
print(f"阶段3结果：{len(df)} 行，{df['imsi'].nunique()} 用户")
print(f"  驻留点：{(df['label']==0).sum()}")
print(f"  出行点：{(df['label']==1).sum()}")

# 读取出行目的静态数据（GBK编码）
purpose_path = os.path.join(DATA_DIR, "出行目的静态数据.csv")
purpose_df = pd.read_csv(purpose_path, encoding="gbk")
print(f"\n出行目的静态数据：{len(purpose_df)} 行")
print(f"  列：{list(purpose_df.columns)}")
print(f"  前5行:")
print(purpose_df.head().to_string(index=False))

# 静态数据中的目的类型分布
print(f"\n目的类型分布:")
print(purpose_df.iloc[:, 0].value_counts())

# 时间规则演示
print(f"\n{'='*60}")
print("时间规则演示")
print(f"{'='*60}")
```

```
def judge_purpose_by_time(start_hour, end_hour, duration_hours):
    """根据时间规则判定出行目的。

    Args:
        start_hour: 开始小时（0-23）
        end_hour: 结束小时（0-23）
        duration_hours: 持续时间（小时）

    Returns:
        purpose: 出行目的（居住/购物/旅游/其他）
    """
    # 居住：持续>4h，且开始或结束在夜间（0-6点或20-24点）
    is_night_start = (start_hour >= 0 and start_hour < 6) or (start_hour >= 20 and
start_hour <= 23)
    is_night_end = (end_hour >= 0 and end_hour < 6) or (end_hour >= 20 and end_hour <=
23)
    if duration_hours > 4 and (is_night_start or is_night_end):
        return "居住"

    # 购物：9点后起，21点前止
    if start_hour >= 9 and end_hour <= 21:
        return "购物"

    # 旅游：9点后起，17点前止
    if start_hour >= 9 and end_hour <= 17:
        return "旅游"

    return "其他"

# 测试用例
test_cases = [
    (22, 6, 8, "夜间在家睡觉"),
    (10, 12, 2, "上午逛商场"),
    (10, 15, 5, "白天旅游"),
    (14, 18, 4, "下午活动"),
    (8, 22, 14, "全天在外"),
]
```

```
print(f"{'开始':<6} {'结束':<6} {'持续(h)':<10} {'场景':<15} {'判定目的':<10}")
print("-" * 55)
for start, end, dur, scene in test_cases:
    purpose = judge_purpose_by_time(start, end, dur)
    print(f"{start:<6} {end:<6} {dur:<10} {scene:<15} {purpose:<10}")

print(f"\n规则说明:")
print(f"  居住: 持续>4h 且 (开始或结束在0-6点或20-24点)")
print(f"  购物: 开始>=9点 且 结束<=21点")
print(f"  旅游: 开始>=9点 且 结束<=17点")
print(f"  其他: 不满足以上条件")
print(f"  优先级: 居住 > 购物 > 旅游 > 其他")
```

1.5 运行结果与解读

运行后展示：
- 阶段3结果统计；
- 出行目的静态数据（GBK编码读取）；
- 时间规则测试用例（5个场景的判定结果）。

> 关键理解：
> 1. 出行目的基于驻留段，不是单个点；
> 2. 时间规则：居住最严格（>4h且夜间），购物和旅游较宽松；
> 3. 优先级：居住 > 购物 > 旅游 > 其他；
> 4. 静态数据为GBK编码，读取时需指定 encoding="gbk"。

1.6 当堂练习

1. 设计3个边界测试用例（如开始=9点、结束=21点、持续=4h），验证规则；
2. 思考：为什么居住的判定需要"持续>4h"？（提示：短时间夜间停留可能是路过，不是居住）
3. 尝试修改购物和旅游的时间阈值，分析对结果的影响。

1.7 课时小结

- 出行目的：居住、购物、旅游、其他四类；
- 时间规则：居住（>4h且夜间）、购物（9-21点）、旅游（9-17点）；
- 优先级：居住 > 购物 > 旅游 > 其他；

- 空间匹配：驻留段中心与静态数据<1km取最近；
- 静态数据GBK编码，读取时指定 encoding="gbk"。

课时2：驻留段识别（连续驻留点合并）

2.1 课时目标

掌握驻留段识别方法：将连续的驻留点合并为驻留段，计算段的开始/结束时间、持续时间、中心位置。

2.2 知识点深度讲解

2.2.1 什么是驻留段

驻留段是用户在一个驻留区域内连续停留的一段时间，由连续的驻留点组成。

例子：
- 用户8:00-8:05在家（3个驻留点）→ 一个驻留段；
- 用户8:10-8:20在上班路上（5个出行点）→ 出行段；
- 用户8:25-12:00在公司（20个驻留点）→ 一个驻留段。

2.2.2 为什么需要驻留段

出行目的是针对"段"的，不是针对"点"的：
- 单个驻留点无法判断持续时间；
- 驻留段有明确的开始时间、结束时间、持续时间，可以应用时间规则；
- 驻留段有中心位置，可以进行空间匹配。

2.2.3 驻留段识别算法

驻留段识别的核心是"连续驻留点合并"：
1. 按用户、时间排序；
2. 遍历每个点，标记是否为驻留点（label=0）；
3. 连续的驻留点合并为一个驻留段；
4. 出行点打断驻留段（出行点不属于任何驻留段）。

实现方法：用 cumsum 实现段ID（与阶段1的连续段识别类似）。

```
# 标记驻留点
df["is_stay"] = df["label"] == 0

# 段ID变化：用户变化 或 驻留状态变化
df["seg_change"] = (df["imsi"] != df["imsi"].shift(1)) | \
                   (df["is_stay"] != df["is_stay"].shift(1))
df["seg_id"] = df.groupby("imsi")["seg_change"].cumsum()

# 只保留驻留段
stay_segments = df[df["is_stay"]].groupby(["imsi", "seg_id"])
```

2.2.4 驻留段属性计算

对每个驻留段，计算：

1. **开始时间**：段内最早的时间；
2. **结束时间**：段内最晚的时间；
3. **持续时间**：结束时间 - 开始时间（小时）；
4. **点数**：段内驻留点数量；
5. **中心经度**：段内所有点经度的均值；
6. **中心纬度**：段内所有点纬度的均值。

```
segment_stats = stay_segments.agg(
    start_time=("time", "min"),
    end_time=("time", "max"),
    points=("time", "count"),
    center_lon=("lon", "mean"),
    center_lat=("lat", "mean"),
).reset_index()

# 持续时间
segment_stats["start_dt"] = pd.to_datetime(segment_stats["start_time"],
format="%Y%m%d%H%M%S")
segment_stats["end_dt"] = pd.to_datetime(segment_stats["end_time"],
format="%Y%m%d%H%M%S")
segment_stats["duration_hours"] = (segment_stats["end_dt"] -
segment_stats["start_dt"]).dt.total_seconds() / 3600
```

2.2.5 驻留段过滤

有些驻留段点数太少或持续时间太短，可能是噪声：
- 点数<2的驻留段（只有1个驻留点）：可能是误判；
- 持续时间<5分钟的驻留段：可能是短暂停留，不是真正的驻留。

本项目不过滤（保持所有驻留段），但可以根据需要添加过滤。

2.3 项目实现步骤

1. 读取阶段3结果；
2. 用 cumsum 实现驻留段ID；
3. 按 imsi-seg_id 分组，计算段属性；
4. 统计驻留段数量和分布；
5. 输出驻留段结果。

2.4 完整代码（驻留段识别）

```
# ============================================================
# 课时2：驻留段识别（连续驻留点合并）
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

# 读取阶段3结果
df = pd.read_csv(os.path.join(RESULT_DIR, "stage3_label.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
df = df.sort_values(["imsi", "time"]).reset_index(drop=True)
print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

# 标记驻留点
df["is_stay"] = df["label"] == 0
print(f"驻留点：{df['is_stay'].sum()} ({df['is_stay'].mean()*100:.1f}%)")

# 驻留段ID：用户变化 或 驻留状态变化 → 新段
df["user_change"] = df["imsi"] != df["imsi"].shift(1)
df["stay_change"] = df["is_stay"] != df["is_stay"].shift(1)
df["seg_change"] = df["user_change"] | df["stay_change"]
df["seg_id"] = df.groupby("imsi")["seg_change"].cumsum()

# 只保留驻留段
stay_df = df[df["is_stay"]].copy()
print(f"驻留段数：{stay_df.groupby(['imsi', 'seg_id']).ngroups}")

# 计算驻留段属性
segments = stay_df.groupby(["imsi", "seg_id"]).agg(
    start_time=("time", "min"),
    end_time=("time", "max"),
    points=("time", "count"),
```

```
    center_lon=("lon", "mean"),
    center_lat=("lat", "mean"),
).reset_index()

# 持续时间
segments["start_dt"] = pd.to_datetime(segments["start_time"], format="%Y%m%d%H%M%S",
errors="coerce")
segments["end_dt"] = pd.to_datetime(segments["end_time"], format="%Y%m%d%H%M%S",
errors="coerce")
segments["duration_hours"] = (segments["end_dt"] -
segments["start_dt"]).dt.total_seconds() / 3600

# 开始/结束小时
segments["start_hour"] = segments["start_dt"].dt.hour
segments["end_hour"] = segments["end_dt"].dt.hour

print(f"\n驻留段统计:")
print(f"  总段数: {len(segments)}")
print(f"  平均点数: {segments['points'].mean():.1f}")
print(f"  平均持续时间: {segments['duration_hours'].mean():.2f} 小时")
print(f"  中位持续时间: {segments['duration_hours'].median():.2f} 小时")

# 持续时间分布
print(f"\n持续时间分布:")
bins = [0, 0.1, 0.5, 1, 2, 4, 8, 12, 24]
labels_bin = ["<6分", "6-30分", "30分-1h", "1-2h", "2-4h", "4-8h", "8-12h", "12-24h"]
segments["duration_bin"] = pd.cut(segments["duration_hours"], bins=bins,
labels=labels_bin, right=False)
print(segments["duration_bin"].value_counts().sort_index())

# 点数分布
print(f"\n点数分布:")
print(f"  1点段: {(segments['points']==1).sum()}")
print(f"  2点段: {(segments['points']==2).sum()}")
print(f"  3-5点段: {((segments['points']>=3)&(segments['points']<=5)).sum()}")
print(f"  6-10点段: {((segments['points']>=6)&(segments['points']<=10)).sum()}")
print(f"  10点以上段: {(segments['points']>10).sum()}")
```

```
# 用户级统计
user_seg = segments.groupby("imsi").size()
print(f"\n用户级统计:")
print(f"  平均驻留段数/用户: {user_seg.mean():.1f}")
print(f"  最多驻留段数: {user_seg.max()}")

# 展示前10个驻留段
print(f"\n前10个驻留段:")
display_cols = ["imsi", "start_time", "end_time", "points", "duration_hours",
                "start_hour", "end_hour", "center_lon", "center_lat"]
print(segments[display_cols].head(10).to_string(index=False))

# 保存驻留段结果
output_path = os.path.join(RESULT_DIR, "stay_segments.csv")
segments.to_csv(output_path, index=False, encoding="utf-8")
print(f"\n保存: {output_path}")
```

2.5 运行结果与解读

运行后展示:
- 驻留段数量和统计（平均点数、平均持续时间）；
- 持续时间分布（6个区间）；
- 点数分布；
- 用户级统计（平均驻留段数）；
- 前10个驻留段详情；
- 保存 stay_segments.csv。

> 关键理解:
> 1. 驻留段由连续驻留点合并而成，出行点打断；
> 2. 用 cumsum 实现段ID（与阶段1连续段识别类似）；
> 3. 大部分驻留段持续时间<4小时（短停留），少数>8小时（可能是居住）；
> 4. 驻留段有开始/结束时间、持续时间、中心位置，可应用时间规则和空间匹配。

2.6 当堂练习

1. 统计持续时间>4小时的驻留段数量，这些可能是居住段；
2. 思考：为什么有些驻留段只有1个点？（提示：可能是误判，或用户只在该位置出现一次）
3. 尝试过滤点数<2的驻留段，对比结果差异。

2.7 课时小结

- 驻留段：连续驻留点合并，出行点打断；
- 段ID用 cumsum 实现；
- 段属性：开始/结束时间、持续时间、点数、中心位置；
- 大部分驻留段持续<4h，少数>8h（可能居住）；
- 输出 stay_segments.csv。

课时3：时间规则判定（居住/购物/旅游）

3.1 课时目标

对每个驻留段应用时间规则，判定出行目的（居住/购物/旅游/其他）。

3.2 知识点深度讲解

3.2.1 时间规则详解

居住:
- 持续时间 > 4小时；
- 开始时间在夜间（0-6点 或 20-24点），或结束时间在夜间；
- 业务含义：用户在家过夜（或住酒店）。

购物:
- 开始时间 >= 9点；
- 结束时间 <= 21点；
- 业务含义：用户在商场营业时间内停留。

旅游:
- 开始时间 >= 9点；
- 结束时间 <= 17点；
- 业务含义：用户在景点开放时间内停留（白天）。

其他:
- 不满足以上任何条件。

3.2.2 判定顺序

按优先级依次判定：
1. 先判居住（最严格）；
2. 不满足居住，判购物；

3. 不满足购物，判旅游；
4. 都不满足，判其他。

```
if is_living:
    purpose = "居住"
elif is_shopping:
    purpose = "购物"
elif is_travel:
    purpose = "旅游"
else:
    purpose = "其他"
```

3.2.3 边界情况

1. **持续时间为0**：开始时间=结束时间（只有1个点），持续时间=0，不满足居住；
2. **跨天**：开始时间23点，结束时间次日1点，持续时间2小时，开始和结束都在夜间，但持续<4h → 不满足居住；
3. **开始=9点**：满足购物和旅游的开始条件，优先判购物（因为购物结束<=21比旅游结束<=17宽松）；
4. **结束=17点**：满足旅游结束条件，也满足购物结束条件，优先判购物。

3.2.4 时间规则的局限性

时间规则只能初步判定，可能误判：
- 用户在公司上班（9-18点），时间规则可能判为"购物"或"旅游"；
- 用户在朋友家做客（20-22点），时间规则可能判为"其他"；
- 用户在医院看病（10-12点），时间规则可能判为"购物"。

空间匹配可以修正部分误判（如公司附近没有商场/景点 → 改为其他）。

3.3 项目实现步骤

1. 读取驻留段结果；
2. 对每个驻留段计算开始小时、结束小时、持续时间；
3. 按优先级应用时间规则；
4. 统计出行目的分布；
5. 输出带时间规则判定的驻留段结果。

3.4 完整代码（时间规则判定）

```
# ============================================================
# 课时3：时间规则判定（居住/购物/旅游）
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

# 读取驻留段
segments = pd.read_csv(os.path.join(RESULT_DIR, "stay_segments.csv"),
                       dtype={"imsi": str, "start_time": str, "end_time": str})
print(f"驻留段：{len(segments)} 段")

# 确保时间字段
segments["start_dt"] = pd.to_datetime(segments["start_time"], format="%Y%m%d%H%M%S",
errors="coerce")
segments["end_dt"] = pd.to_datetime(segments["end_time"], format="%Y%m%d%H%M%S",
errors="coerce")
segments["start_hour"] = segments["start_dt"].dt.hour
segments["end_hour"] = segments["end_dt"].dt.hour
segments["duration_hours"] = (segments["end_dt"] -
segments["start_dt"]).dt.total_seconds() / 3600


def judge_purpose_by_time(row):
    """根据时间规则判定出行目的。"""
    start_hour = row["start_hour"]
    end_hour = row["end_hour"]
    duration = row["duration_hours"]

    # 夜间判定
    is_night_start = (0 <= start_hour < 6) or (20 <= start_hour <= 23)
    is_night_end = (0 <= end_hour < 6) or (20 <= end_hour <= 23)
```

```
    # 居住：持续>4h 且 (开始或结束在夜间)
    if duration > 4 and (is_night_start or is_night_end):
        return "居住"

    # 购物：9点后起，21点前止
    if start_hour >= 9 and end_hour <= 21:
        return "购物"

    # 旅游：9点后起，17点前止
    if start_hour >= 9 and end_hour <= 17:
        return "旅游"

    return "其他"


# 应用时间规则
segments["purpose_by_time"] = segments.apply(judge_purpose_by_time, axis=1)

# 统计
print(f"\n时间规则判定结果:")
print(segments["purpose_by_time"].value_counts())
print(f"\n比例:")
print((segments["purpose_by_time"].value_counts(normalize=True) * 100).round(1))

# 按目的统计持续时间
print(f"\n按目的统计持续时间:")
print(f"{'目的':<8} {'段数':<8} {'平均持续(h)':<12} {'中位持续(h)':<12} {'平均点数':
<10}")
print("-" * 55)
for purpose in ["居住", "购物", "旅游", "其他"]:
    sub = segments[segments["purpose_by_time"] == purpose]
    if len(sub) > 0:
        print(f"{purpose:<8} {len(sub):<8} {sub['duration_hours'].mean():<12.2f} "
              f"{sub['duration_hours'].median():<12.2f} {sub['points'].mean():<10.1f}")

# 居住段详细分析
print(f"\n居住段详细分析:")
```

```
living = segments[segments["purpose_by_time"] == "居住"]
print(f"  居住段数: {len(living)}")
print(f"  涉及用户数: {living['imsi'].nunique()}")
print(f"  持续时间范围: {living['duration_hours'].min():.2f} -
{living['duration_hours'].max():.2f} h")
print(f"  开始时间分布:")
print(living["start_hour"].value_counts().sort_index())

# 边界情况分析
print(f"\n边界情况分析:")
# 持续时间=0（单段点）
zero_dur = segments[segments["duration_hours"] == 0]
print(f"  持续时间=0的段: {len(zero_dur)} (全部判为
{zero_dur['purpose_by_time'].unique().tolist()})")
# 开始=9点
start_9 = segments[segments["start_hour"] == 9]
print(f"  开始=9点的段: {len(start_9)} (购物优先:{(start_9['purpose_by_time']=='购
物').sum()})")
# 跨天（开始>结束小时，且持续>12h）
cross_day = segments[(segments["start_hour"] > segments["end_hour"]) &
(segments["duration_hours"] > 12)]
print(f"  跨天段: {len(cross_day)} (居住:{(cross_day['purpose_by_time']=='居
住').sum()})")

# 展示各类目的示例
print(f"\n各类目的示例（各3个）:")
for purpose in ["居住", "购物", "旅游", "其他"]:
    sub = segments[segments["purpose_by_time"] == purpose].head(3)
    print(f"\n  【{purpose}】")
    print(sub[["imsi", "start_time", "end_time", "duration_hours", "start_hour",
"end_hour"]].to_string(index=False))

# 保存
output_path = os.path.join(RESULT_DIR, "stay_segments_with_purpose_time.csv")
segments.to_csv(output_path, index=False, encoding="utf-8")
print(f"\n保存: {output_path}")
```

3.5 运行结果与解读

运行后展示：
- 时间规则判定结果（四类目的的数量和比例）；
- 按目的统计持续时间和点数；
- 居住段详细分析（涉及用户数、持续时间范围、开始时间分布）；
- 边界情况分析（持续=0、开始=9点、跨天）；
- 各类目的示例；
- 保存 stay_segments_with_purpose_time.csv。

> 关键理解：
> 1. 时间规则判定后，"其他"通常最多（很多驻留段不满足严格条件）；
> 2. 居住段持续时间最长（>4h），涉及用户数较少；
> 3. 购物和旅游段持续时间较短，开始时间在白天；
> 4. 时间规则可能误判（如公司上班被判为购物），需要空间匹配修正。

3.6 当堂练习

1. 统计"购物"段中持续时间>2h的数量，这些可能是真正的购物（长时间逛商场）；
2. 思考：为什么"其他"类通常最多？（提示：时间规则较严格，很多短停留不满足条件）
3. 尝试放宽居住条件（持续>2h），对比居住段数量变化。

3.7 课时小结

- 时间规则：居住（>4h且夜间）、购物（9-21点）、旅游（9-17点）、其他；
- 判定顺序：居住 > 购物 > 旅游 > 其他；
- 居住段持续最长，涉及用户较少；
- 购物/旅游段持续较短，开始在白天；
- 时间规则可能误判，需要空间匹配修正。

课时4：出行目的静态数据读取与GBK解码

4.1 课时目标

掌握出行目的静态数据的读取方法（GBK编码），理解数据结构和预处理。

4.2 知识点深度讲解

4.2.1 静态数据文件

出行目的静态数据文件：`出行目的静态数据.csv`
- 编码：GBK（不是UTF-8）；
- 列数：4列；
- 行数：约1273行；
- 内容：商场、景点等POI（Point of Interest）位置信息。

为什么是GBK编码？
- 很多中文CSV文件用GBK编码（Excel默认保存为GBK）；
- 如果用UTF-8读取，会出现乱码或报错；
- 读取时必须指定 `encoding="gbk"`。

4.2.2 GBK vs UTF-8

编码	特点	中文字节数	适用场景
UTF-8	国际标准，可变长	3字节	互联网、编程
GBK	国家标准，固定长	2字节	中文Windows、Excel

读取GBK文件：

```
df = pd.read_csv("file.csv", encoding="gbk")
```

如果不确定编码：

```
# 尝试多种编码
for enc in ["utf-8", "gbk", "gb2312", "latin1"]:
    try:
        df = pd.read_csv("file.csv", encoding=enc)
        print(f"成功用 {enc} 读取")
        break
    except UnicodeDecodeError:
        continue
```

4.2.3 静态数据结构

出行目的静态数据有4列（具体列名需读取后确认）：
1. **目的类型**：商场/景点等分类；

2. **名称**：POI名称；
3. **经度**：POI经度；
4. **纬度**：POI纬度。

> 注意：不同项目的静态数据列名可能不同，读取后需用 `df.columns` 确认。

4.2.4 静态数据预处理

1. **列名重命名**：统一为 `type, name, lon, lat`；
2. **经纬度类型转换**：确保为数值型；
3. **去重**：去除重复的POI；
4. **过滤无效值**：去除经纬度为空或异常的行。

4.2.5 目的类型映射

静态数据中的目的类型可能有多种名称，需要映射为标准类型：
- "购物中心"、"商场"、"超市" → "购物"；
- "景点"、"公园"、"风景区" → "旅游"；
- 其他 → "其他"。

本项目直接用静态数据中的类型，不做映射（或简单映射）。

4.3 项目实现步骤

1. 用GBK编码读取出行目的静态数据；
2. 查看数据结构（列名、行数、前几行）；
3. 预处理（列名重命名、类型转换、去重）；
4. 统计目的类型分布；
5. 保存预处理后的静态数据。

4.4 完整代码（静态数据读取与GBK解码）

```
# ============================================================
# 课时4：出行目的静态数据读取与GBK解码
# ============================================================

import pandas as pd
import numpy as np
import os

DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统\09.交通时空大数据分析系统\B-项目数据"
RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

# 读取出行目的静态数据（GBK编码）
purpose_path = os.path.join(DATA_DIR, "出行目的静态数据.csv")

print("=" * 60)
print("读取出行目的静态数据")
print("=" * 60)

# 方法1：直接指定gbk
try:
    purpose_df = pd.read_csv(purpose_path, encoding="gbk")
    print(f"成功用 GBK 读取")
except UnicodeDecodeError:
    print("GBK 读取失败，尝试其他编码...")
    for enc in ["utf-8", "gb2312", "latin1"]:
        try:
            purpose_df = pd.read_csv(purpose_path, encoding=enc)
            print(f"成功用 {enc} 读取")
            break
        except UnicodeDecodeError:
            continue

print(f"  行数：{len(purpose_df)}")
print(f"  列数：{len(purpose_df.columns)}")
```

```
print(f"  列名: {list(purpose_df.columns)}")
print(f"\n前10行:")
print(purpose_df.head(10).to_string(index=False))

# 数据类型
print(f"\n数据类型:")
print(purpose_df.dtypes)

# 缺失值
print(f"\n缺失值统计:")
print(purpose_df.isnull().sum())

# 第一列（目的类型）分布
print(f"\n第一列（目的类型）分布:")
print(purpose_df.iloc[:, 0].value_counts())

# 预处理
print(f"\n{'='*60}")
print("数据预处理")
print(f"{'='*60}")

# 假设列顺序为：类型、名称、经度、纬度（根据实际数据调整）
# 重命名列
if len(purpose_df.columns) == 4:
    purpose_df.columns = ["poi_type", "poi_name", "lon", "lat"]
    print(f"列重命名为: {list(purpose_df.columns)}")
else:
    print(f"列数不是4，保持原列名: {list(purpose_df.columns)}")

# 经纬度转换为数值型
purpose_df["lon"] = pd.to_numeric(purpose_df["lon"], errors="coerce")
purpose_df["lat"] = pd.to_numeric(purpose_df["lat"], errors="coerce")

# 去除经纬度缺失的行
before = len(purpose_df)
purpose_df = purpose_df.dropna(subset=["lon", "lat"])
after = len(purpose_df)
print(f"去除经纬度缺失: {before} -> {after} (删除{before-after}行)")
```

```
# 去重
before = len(purpose_df)
purpose_df = purpose_df.drop_duplicates()
after = len(purpose_df)
print(f"去重: {before} -> {after} (删除{before-after}行)")

# 经纬度范围检查
print(f"\n经纬度范围:")
print(f"  经度: {purpose_df['lon'].min():.4f} ~ {purpose_df['lon'].max():.4f}")
print(f"  纬度: {purpose_df['lat'].min():.4f} ~ {purpose_df['lat'].max():.4f}")

# 目的类型分布（预处理后）
print(f"\n预处理后目的类型分布:")
print(purpose_df["poi_type"].value_counts())

# 保存预处理后的静态数据
output_path = os.path.join(RESULT_DIR, "purpose_poi_cleaned.csv")
purpose_df.to_csv(output_path, index=False, encoding="utf-8")
print(f"\n保存: {output_path}")
print(f"  行数: {len(purpose_df)}")

# 同时读取出行方式静态数据（对比）
print(f"\n{'='*60}")
print("对比：出行方式静态数据（也是GBK）")
print(f"{'='*60}")
mode_path = os.path.join(DATA_DIR, "出行方式静态数据.csv")
mode_df = pd.read_csv(mode_path, encoding="gbk")
print(f"  行数: {len(mode_df)}")
print(f"  列名: {list(mode_df.columns)}")
print(f"  前5行:")
print(mode_df.head().to_string(index=False))

print(f"\n结论：")
print(f"  1. 出行目的静态数据为GBK编码，读取时必须指定 encoding='gbk'")
print(f"  2. 数据包含POI类型、名称、经度、纬度4列")
print(f"  3. 预处理后保存为UTF-8编码，方便后续使用")
```

4.5 运行结果与解读

运行后展示：
- 静态数据读取结果（行数、列名、前10行）；
- 数据类型和缺失值统计；
- 目的类型分布；
- 预处理（列重命名、类型转换、去重、缺失值处理）；
- 经纬度范围检查；
- 出行方式静态数据对比（也是GBK编码）。

> 关键理解：
> 1. 出行目的静态数据为GBK编码，读取时必须指定 `encoding="gbk"` ；
> 2. 数据包含POI类型、名称、经度、纬度4列；
> 3. 预处理后保存为UTF-8编码，方便后续空间匹配；
> 4. 出行方式静态数据也是GBK编码，后续课时会用到。

4.6 当堂练习

1. 尝试用UTF-8编码读取静态数据，观察报错信息；
2. 统计静态数据中各类型POI的经纬度范围，分析是否都在沈阳市内；
3. 思考：为什么静态数据用GBK而不是UTF-8？（提示：Excel默认保存为GBK）

4.7 课时小结

- 静态数据GBK编码，读取时指定 `encoding="gbk"` ；
- 数据结构：4列（类型、名称、经度、纬度），约1273行；
- 预处理：列重命名、类型转换、去重、缺失值处理；
- 预处理后保存为UTF-8编码；
- 出行方式静态数据也是GBK编码。

课时5：空间匹配（<1km取最近）

5.1 课时目标

掌握空间匹配方法：将驻留段中心与出行目的静态数据（POI）匹配，<1km取最近，验证和修正时间规则的判定。

5.2 知识点深度讲解

5.2.1 空间匹配的目的

时间规则只能初步判定出行目的，可能误判（如公司上班被判为购物）。空间匹配可以验证和修正：
- 如果时间规则判为"购物"，且附近1km内有商场 → 确认购物；
- 如果时间规则判为"旅游"，且附近1km内有景点 → 确认旅游；
- 如果时间规则判为"购物"，但附近1km内没有商场 → 可能是误判，改为"其他"；
- 居住不需要空间匹配（家的位置不在静态数据中）。

5.2.2 空间匹配算法

对每个驻留段：
1. 计算驻留段中心到所有POI的距离（Haversine）；
2. 找到距离最近的POI；
3. 如果最近距离<1km，匹配成功，记录最近POI的信息；
4. 如果最近距离>=1km，匹配失败。

```
# 驻留段中心
seg_lon, seg_lat = segment["center_lon"], segment["center_lat"]

# 到所有POI的距离
dists = haversine(seg_lon, seg_lat, poi_df["lon"], poi_df["lat"], unit="m")

# 最近POI
min_idx = dists.idxmin()
min_dist = dists.min()

if min_dist < 1000:
    # 匹配成功
    nearest_poi = poi_df.loc[min_idx]
else:
    # 匹配失败
    nearest_poi = None
```

5.2.3 <1km 阈值的业务含义

为什么是1km?
- 基站覆盖范围：城市内基站覆盖半径约200-500m；

- 驻留段中心是多个基站位置的均值，可能有几百米误差；
- 1km 是合理的匹配半径（允许一定的定位误差）。

阈值的影响：
- 阈值太小（如100m）：匹配成功率低，很多驻留段匹配不到POI；
- 阈值太大（如5km）：匹配成功率高，但可能匹配到错误的POI（附近有多个POI时）；
- 1km 是合理的平衡点。

5.2.4 匹配结果与时间规则的结合

空间匹配后，最终出行目的的判定逻辑：
1. **居住**：时间规则判为居住 → 保持居住（不需要空间匹配）；
2. **购物**：时间规则判为购物 且 空间匹配到商场 → 购物；时间规则判为购物 但 没匹配到商场 → 其他；
3. **旅游**：时间规则判为旅游 且 空间匹配到景点 → 旅游；时间规则判为旅游 但 没匹配到景点 → 其他；
4. **其他**：保持其他。

> 注意：本项目简化处理，空间匹配只用于"验证"时间规则的判定，不改变时间规则的判定。即时间规则判为购物，即使没匹配到商场，也保持购物。具体实现可根据项目要求调整。

5.2.5 性能优化

如果驻留段数量多（如几千段），POI数量多（如几千个），双重循环的时间复杂度是 O(N×M)，可能较慢。

优化方法：
1. **向量化**：用 numpy 广播一次计算一个驻留段到所有POI的距离（已实现）；
2. **空间索引**：用 KDTree 或 BallTree 加速最近邻搜索（sklearn.neighbors）；
3. **粗筛**：先用经纬度范围粗筛（如经度差<0.01度≈1km），再精确计算距离。

本项目数据量不大（驻留段约1000段，POI约1273个），向量化计算足够快。

5.3 项目实现步骤

1. 读取驻留段结果（含时间规则判定）和POI静态数据；
2. 对每个驻留段计算到所有POI的距离；
3. 找到最近POI，判断是否<1km；
4. 记录匹配结果（最近POI名称、类型、距离）；
5. 结合时间规则，给出最终出行目的；
6. 统计匹配成功率和最终目的分布。

5.4 完整代码（空间匹配）

```
# ============================================================
# 课时5：空间匹配（<1km取最近）
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"


def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """球面距离（向量化）。"""
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c * (1000 if unit == "m" else 1)


# 读取驻留段（含时间规则判定）
segments = pd.read_csv(os.path.join(RESULT_DIR, "stay_segments_with_purpose_time.csv"),
                       dtype={"imsi": str, "start_time": str, "end_time": str})
print(f"驻留段：{len(segments)} 段")

# 读取POI静态数据
poi_df = pd.read_csv(os.path.join(RESULT_DIR, "purpose_poi_cleaned.csv"))
print(f"POI数量：{len(poi_df)}")
print(f"POI类型：{poi_df['poi_type'].unique().tolist()}")

# 空间匹配
MATCH_THRESHOLD = 1000  # 匹配阈值（m）
```

```
print(f"\n开始空间匹配（阈值={MATCH_THRESHOLD}m）...")

# 初始化匹配结果列
segments["nearest_poi_name"] = None
segments["nearest_poi_type"] = None
segments["nearest_dist_m"] = np.nan
segments["match_success"] = False

# POI经纬度数组（向量化计算）
poi_lons = poi_df["lon"].values
poi_lats = poi_df["lat"].values

for idx, seg in segments.iterrows():
    # 驻留段中心
    seg_lon = seg["center_lon"]
    seg_lat = seg["center_lat"]

    # 到所有POI的距离（向量化）
    dists = haversine(seg_lon, seg_lat, poi_lons, poi_lats, unit="m")

    # 最近POI
    min_idx = np.argmin(dists)
    min_dist = dists[min_idx]

    if min_dist < MATCH_THRESHOLD:
        # 匹配成功
        nearest_poi = poi_df.iloc[min_idx]
        segments.at[idx, "nearest_poi_name"] = nearest_poi["poi_name"]
        segments.at[idx, "nearest_poi_type"] = nearest_poi["poi_type"]
        segments.at[idx, "nearest_dist_m"] = min_dist
        segments.at[idx, "match_success"] = True

    # 进度
    if (idx + 1) % 200 == 0:
        print(f"  已处理 {idx+1}/{len(segments)} 段...")

# 匹配统计
print(f"\n匹配统计:")
```

```
print(f"  匹配成功: {segments['match_success'].sum()}
({segments['match_success'].mean()*100:.1f}%)")
print(f"  匹配失败: {(~segments['match_success']).sum()}
({(~segments['match_success']).mean()*100:.1f}%)")

# 匹配距离统计
matched = segments[segments["match_success"]]
print(f"\n匹配距离统计:")
print(f"  最小: {matched['nearest_dist_m'].min():.1f} m")
print(f"  最大: {matched['nearest_dist_m'].max():.1f} m")
print(f"  中位数: {matched['nearest_dist_m'].median():.1f} m")
print(f"  平均值: {matched['nearest_dist_m'].mean():.1f} m")

# 按时间规则目的统计匹配率
print(f"\n按时间规则目的统计匹配率:")
print(f"{'时间规则目的':<12} {'段数':<8} {'匹配成功':<10} {'匹配率':<10}")
print("-" * 45)
for purpose in ["居住", "购物", "旅游", "其他"]:
    sub = segments[segments["purpose_by_time"] == purpose]
    if len(sub) > 0:
        match_rate = sub["match_success"].mean() * 100
        print(f"{purpose:<12} {len(sub):<8} {sub['match_success'].sum():<10}
{match_rate:<10.1f}%")

# 最终出行目的（时间规则 + 空间匹配验证）
# 简化处理：保持时间规则的判定，空间匹配结果作为参考
segments["final_purpose"] = segments["purpose_by_time"]

# 最终目的分布
print(f"\n最终出行目的分布:")
print(segments["final_purpose"].value_counts())
print(f"\n比例:")
print((segments["final_purpose"].value_counts(normalize=True) * 100).round(1))

# 展示匹配成功的示例
print(f"\n匹配成功的示例（前10个）:")
matched_sample = matched[["imsi", "purpose_by_time", "nearest_poi_name",
                          "nearest_poi_type", "nearest_dist_m"]].head(10)
```

```
print(matched_sample.to_string(index=False))


# 保存
output_path = os.path.join(RESULT_DIR, "stay_segments_with_match.csv")
segments.to_csv(output_path, index=False, encoding="utf-8")
print(f"\n保存: {output_path}")
```

5.5 运行结果与解读

运行后展示:
- 匹配统计（成功率、失败率）；
- 匹配距离统计（最小、最大、中位、平均）；
- 按时间规则目的统计匹配率;
- 最终出行目的分布;
- 匹配成功的示例;
- 保存 stay_segments_with_match.csv。

> 关键理解:
> 1. 空间匹配成功率约50%-70%（很多驻留段附近没有POI）；
> 2. 匹配距离中位数约200-500m（大部分匹配到的POI很近）；
> 3. 购物和旅游段的匹配率较高（附近有商场/景点）；
> 4. 居住段匹配率较低（家附近可能没有商场/景点，或匹配到的POI类型不匹配）。

5.6 当堂练习

1. 把匹配阈值改为500m和2000m，分别统计匹配率，分析哪个更合理;
2. 思考：为什么居住段的匹配率较低？（提示：家附近可能没有商场/景点）
3. 尝试实现"空间匹配修正"：时间规则判为购物但没匹配到商场 → 改为其他，对比结果差异。

5.7 课时小结

- 空间匹配：驻留段中心与POI<1km取最近;
- 匹配算法：向量化计算距离，argmin找最近;
- 1km阈值：允许定位误差，合理平衡点;
- 匹配成功率约50%-70%，匹配距离中位200-500m;
- 购物/旅游段匹配率高，居住段匹配率低;
- 最终目的保持时间规则判定，空间匹配作为验证。

课时6：出行目的完整实现（purpose.py）

6.1 课时目标

将出行目的识别所有逻辑整合为完整模块 purpose.py。

6.2 知识点深度讲解

6.2.1 模块结构

purpose.py 的结构：
1. 配置区（路径、阈值）；
2. 工具函数（haversine、时间规则判定）；
3. 主函数 purpose_analysis()；
4. 入口。

6.2.2 完整流程

```
读取 stage3_label.csv
  ↓
驻留段识别（连续驻留点合并）
  ↓
时间规则判定（居住/购物/旅游/其他）
  ↓
读取出行目的静态数据（GBK）
  ↓
空间匹配（<1km取最近）
  ↓
输出 purpose_result.csv
```

6.3 项目实现步骤

按上述流程编写完整代码。

6.4 完整代码（purpose.py）

```
# ============================================================
# 出行目的模块：purpose.py
# 功能：驻留段识别、时间规则判定、空间匹配
# 运行：python purpose.py
# 输入：stage3_label.csv，出行目的静态数据.csv
# 输出：purpose_result.csv
# ============================================================

import os
import pandas as pd
import numpy as np

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统\09.交通时空大数据分析系统\B-项目数据"
os.makedirs(RESULT_DIR, exist_ok=True)

# 阈值
MATCH_THRESHOLD = 1000  # 空间匹配阈值（m）


def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """球面距离（向量化）。"""
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    d = R * c
    return d * 1000 if unit == "m" else d


def judge_purpose_by_time(start_hour, end_hour, duration_hours):
    """根据时间规则判定出行目的。"""
```

```
    is_night_start = (0 <= start_hour < 6) or (20 <= start_hour <= 23)
    is_night_end = (0 <= end_hour < 6) or (20 <= end_hour <= 23)
    if duration_hours > 4 and (is_night_start or is_night_end):
        return "居住"
    if start_hour >= 9 and end_hour <= 21:
        return "购物"
    if start_hour >= 9 and end_hour <= 17:
        return "旅游"
    return "其他"


def purpose_analysis():
    """执行出行目的识别完整流程。"""
    print("=" * 60)
    print("出行目的识别")
    print("=" * 60)

    # 步骤1：读取阶段3结果
    print("\n步骤1：读取阶段3结果")
    label_path = os.path.join(RESULT_DIR, "stage3_label.csv")
    if not os.path.exists(label_path):
        print("错误：请先运行 tr_stage3.py")
        return None
    df = pd.read_csv(label_path, dtype={"imsi": str, "time": str,
                                        "lac_id": str, "cell_id": str})
    df = df.sort_values(["imsi", "time"]).reset_index(drop=True)
    print(f"  数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

    # 步骤2：驻留段识别
    print("\n步骤2：驻留段识别")
    df["is_stay"] = df["label"] == 0
    df["user_change"] = df["imsi"] != df["imsi"].shift(1)
    df["stay_change"] = df["is_stay"] != df["is_stay"].shift(1)
    df["seg_change"] = df["user_change"] | df["stay_change"]
    df["seg_id"] = df.groupby("imsi")["seg_change"].cumsum()

    stay_df = df[df["is_stay"]].copy()
    segments = stay_df.groupby(["imsi", "seg_id"]).agg(
```

```
        start_time=("time", "min"),
        end_time=("time", "max"),
        points=("time", "count"),
        center_lon=("lon", "mean"),
        center_lat=("lat", "mean"),
    ).reset_index()
    segments["start_dt"] = pd.to_datetime(segments["start_time"],
format="%Y%m%d%H%M%S", errors="coerce")
    segments["end_dt"] = pd.to_datetime(segments["end_time"], format="%Y%m%d%H%M%S",
errors="coerce")
    segments["duration_hours"] = (segments["end_dt"] -
segments["start_dt"]).dt.total_seconds() / 3600
    segments["start_hour"] = segments["start_dt"].dt.hour
    segments["end_hour"] = segments["end_dt"].dt.hour
    print(f"  驻留段数: {len(segments)}")

    # 步骤3：时间规则判定
    print("\n步骤3：时间规则判定")
    segments["purpose_by_time"] = segments.apply(
        lambda row: judge_purpose_by_time(row["start_hour"], row["end_hour"],
row["duration_hours"]),
        axis=1
    )
    print(f"  时间规则判定结果:")
    print(segments["purpose_by_time"].value_counts().to_string())

    # 步骤4：读取POI静态数据
    print("\n步骤4：读取出行目的静态数据（GBK）")
    poi_path = os.path.join(DATA_DIR, "出行目的静态数据.csv")
    poi_df = pd.read_csv(poi_path, encoding="gbk")
    if len(poi_df.columns) == 4:
        poi_df.columns = ["poi_type", "poi_name", "lon", "lat"]
    poi_df["lon"] = pd.to_numeric(poi_df["lon"], errors="coerce")
    poi_df["lat"] = pd.to_numeric(poi_df["lat"], errors="coerce")
    poi_df = poi_df.dropna(subset=["lon", "lat"]).drop_duplicates()
    print(f"  POI数量: {len(poi_df)}")

    # 步骤5：空间匹配
```

```
    print("\n步骤5：空间匹配（<1km取最近）")
    segments["nearest_poi_name"] = None
    segments["nearest_poi_type"] = None
    segments["nearest_dist_m"] = np.nan
    segments["match_success"] = False

    poi_lons = poi_df["lon"].values
    poi_lats = poi_df["lat"].values

    for idx, seg in segments.iterrows():
        dists = haversine(seg["center_lon"], seg["center_lat"], poi_lons, poi_lats,
unit="m")
        min_idx = np.argmin(dists)
        min_dist = dists[min_idx]
        if min_dist < MATCH_THRESHOLD:
            nearest_poi = poi_df.iloc[min_idx]
            segments.at[idx, "nearest_poi_name"] = nearest_poi["poi_name"]
            segments.at[idx, "nearest_poi_type"] = nearest_poi["poi_type"]
            segments.at[idx, "nearest_dist_m"] = min_dist
            segments.at[idx, "match_success"] = True

    print(f"  匹配成功: {segments['match_success'].sum()}
({segments['match_success'].mean()*100:.1f}%)")

    # 步骤6：最终出行目的
    print("\n步骤6：最终出行目的")
    segments["final_purpose"] = segments["purpose_by_time"]
    print(f"  最终目的分布:")
    print(segments["final_purpose"].value_counts().to_string())

    # 输出
    output_path = os.path.join(RESULT_DIR, "purpose_result.csv")
    output_cols = ["imsi", "start_time", "end_time", "duration_hours", "points",
                   "center_lon", "center_lat", "purpose_by_time",
                   "nearest_poi_name", "nearest_poi_type", "nearest_dist_m",
                   "match_success", "final_purpose"]
    segments[output_cols].to_csv(output_path, index=False, encoding="utf-8")
    print(f"\n输出: {output_path}")
```

```
    print(f"  行数: {len(segments)}")

    print("\n" + "=" * 60)
    print("出行目的识别完成！")
    print("=" * 60)
    return segments


if __name__ == "__main__":
    purpose_analysis()
```

6.5 运行结果与解读

运行后展示每步统计，输出 purpose_result.csv。

6.6 当堂练习

1. 运行 purpose.py，记录最终目的分布和匹配率；
2. 检查 purpose_result.csv 的前20行，理解各字段含义。

6.7 课时小结

- purpose.py 整合出行目的所有逻辑；
- 6步流程：读取数据→驻留段识别→时间规则→读取POI→空间匹配→输出；
- 输出 purpose_result.csv。

课时7：出行目的结果分析与可视化

7.1 课时目标

分析出行目的结果，可视化目的分布、时间分布、空间分布。

7.2 完整代码（结果分析与可视化）

```
# ============================================================
# 课时7：出行目的结果分析与可视化
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

df = pd.read_csv(os.path.join(RESULT_DIR, "purpose_result.csv"),
                 dtype={"imsi": str, "start_time": str, "end_time": str})
print(f"驻留段：{len(df)} 段")

# 图1：出行目的分布（饼图+柱状图）
print("生成图1：出行目的分布")
purpose_counts = df["final_purpose"].value_counts()
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].pie(purpose_counts.values, labels=purpose_counts.index, autopct="%1.1f%%",
            colors=["green", "blue", "orange", "gray"], startangle=90)
axes[0].set_title("出行目的分布（饼图）")
axes[1].bar(purpose_counts.index, purpose_counts.values,
            color=["green", "blue", "orange", "gray"], edgecolor="black")
axes[1].set_ylabel("驻留段数")
axes[1].set_title("出行目的分布（柱状图）")
for i, v in enumerate(purpose_counts.values):
    axes[1].text(i, v + 5, str(v), ha="center")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_purpose_dist.png"), dpi=150,
```

```
bbox_inches="tight")
plt.close()

# 图2：各目的持续时间分布
print("生成图2：各目的持续时间分布")
fig, ax = plt.subplots(figsize=(10, 6))
purposes = ["居住", "购物", "旅游", "其他"]
colors = ["green", "blue", "orange", "gray"]
data = [df[df["final_purpose"] == p]["duration_hours"].values for p in purposes]
bp = ax.boxplot(data, labels=purposes, patch_artist=True)
for patch, color in zip(bp["boxes"], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
ax.set_ylabel("持续时间（小时）")
ax.set_title("各出行目的持续时间分布")
ax.set_yscale("log")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_purpose_duration.png"), dpi=150,
bbox_inches="tight")
plt.close()

# 图3：各目的开始时间分布
print("生成图3：各目的开始时间分布")
df["start_hour"] = pd.to_datetime(df["start_time"], format="%Y%m%d%H%M%S",
errors="coerce").dt.hour
fig, ax = plt.subplots(figsize=(12, 5))
for purpose, color in zip(purposes, colors):
    sub = df[df["final_purpose"] == purpose]
    hour_dist = sub["start_hour"].value_counts().reindex(range(24), fill_value=0)
    ax.plot(hour_dist.index, hour_dist.values, label=purpose, color=color, linewidth=2)
ax.set_xlabel("开始小时")
ax.set_ylabel("驻留段数")
ax.set_title("各出行目的开始时间分布")
ax.set_xticks(range(24))
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_purpose_start_hour.png"), dpi=150,
```

```
bbox_inches="tight")
plt.close()

# 图4：空间分布
print("生成图4：出行目的空间分布")
fig, ax = plt.subplots(figsize=(12, 8))
for purpose, color in zip(purposes, colors):
    sub = df[df["final_purpose"] == purpose]
    ax.scatter(sub["center_lon"], sub["center_lat"], c=color, s=10, alpha=0.5,
label=purpose)
ax.set_xlabel("经度"); ax.set_ylabel("纬度")
ax.set_title("出行目的空间分布")
ax.legend(markerscale=3)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_purpose_spatial.png"), dpi=150,
bbox_inches="tight")
plt.close()

print(f"\n图片保存在：{FIG_DIR}")
print(f"  fig_purpose_dist.png")
print(f"  fig_purpose_duration.png")
print(f"  fig_purpose_start_hour.png")
print(f"  fig_purpose_spatial.png")

# 统计摘要
print(f"\n{'='*60}")
print("出行目的统计摘要")
print(f"{'='*60}")
print(f"总驻留段：{len(df)}")
for purpose in purposes:
    sub = df[df["final_purpose"] == purpose]
    print(f"  {purpose}: {len(sub)} ({len(sub)/len(df)*100:.1f}%), "
          f"平均持续{sub['duration_hours'].mean():.2f}h, "
          f"涉及用户{sub['imsi'].nunique()}")
```

7.3 运行结果与解读

生成4张图：目的分布、持续时间、开始时间、空间分布。

7.4 课时小结

- 出行目的分布：其他最多，居住/购物/旅游较少；
- 持续时间：居住最长，购物/旅游较短；
- 开始时间：居住在夜间，购物/旅游在白天；
- 空间分布：各目的聚集在不同区域。

课时8：参数调优与边界情况

8.1 课时目标

理解出行目的识别各参数的影响，处理边界情况。

8.2 知识点深度讲解

8.2.1 参数敏感性

1. **居住持续时间阈值**（默认4h）：影响居住段数量；
2. **购物/旅游时间阈值**（9-21/9-17）：影响购物/旅游段数量；
3. **空间匹配阈值**（默认1km）：影响匹配成功率；
4. **驻留段最小点数**（默认不过滤）：影响驻留段数量。

8.2.2 边界情况

1. **持续时间=0**（单段点）：不满足居住，可能判为购物/旅游/其他；
2. **跨天段**：开始23点，结束次日1点，持续2h，开始和结束都在夜间但持续<4h → 不满足居住；
3. **匹配到错误类型POI**：时间规则判为购物，但匹配到景点 → 以时间规则为准；
4. **POI数据缺失**：某些区域没有POI数据 → 匹配失败。

8.3 完整代码（参数调优与边界情况）

```
# ============================================================
# 课时8：参数调优与边界情况
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

df = pd.read_csv(os.path.join(RESULT_DIR, "purpose_result.csv"),
                 dtype={"imsi": str, "start_time": str, "end_time": str})
df["start_hour"] = pd.to_datetime(df["start_time"], format="%Y%m%d%H%M%S",
errors="coerce").dt.hour
df["end_hour"] = pd.to_datetime(df["end_time"], format="%Y%m%d%H%M%S",
errors="coerce").dt.hour

# 1. 居住持续时间阈值敏感性
print("=" * 60)
print("1. 居住持续时间阈值敏感性")
print("=" * 60)
for threshold in [2, 3, 4, 5, 6, 8]:
    is_night = ((df["start_hour"] >= 0) & (df["start_hour"] < 6)) | \
               ((df["start_hour"] >= 20) & (df["start_hour"] <= 23)) | \
               ((df["end_hour"] >= 0) & (df["end_hour"] < 6)) | \
               ((df["end_hour"] >= 20) & (df["end_hour"] <= 23))
    living = (df["duration_hours"] > threshold) & is_night
    print(f"  阈值>{threshold}h: 居住段={living.sum()} ({living.mean()*100:.1f}%)")

# 2. 空间匹配阈值敏感性
print(f"\n{'='*60}")
print("2. 空间匹配阈值敏感性")
print(f"{'='*60}")
for threshold in [200, 500, 800, 1000, 1500, 2000, 3000]:
    matched = df["nearest_dist_m"] < threshold
```

```
    print(f"  阈值<{threshold}m: 匹配成功={matched.sum()} ({matched.mean()*100:.1f}%)")

# 3. 边界情况分析
print(f"\n{'='*60}")
print("3. 边界情况分析")
print(f"{'='*60}")

# 持续时间=0
zero_dur = df[df["duration_hours"] == 0]
print(f"  持续时间=0的段: {len(zero_dur)}")
print(f"    目的分布: {zero_dur['final_purpose'].value_counts().to_dict()}")

# 跨天段
cross_day = df[(df["start_hour"] > df["end_hour"]) & (df["duration_hours"] > 0)]
print(f"  跨天段: {len(cross_day)}")
print(f"    目的分布: {cross_day['final_purpose'].value_counts().to_dict()}")

# 匹配到错误类型POI（时间规则判购物，匹配到旅游类POI）
# （简化分析，不做严格类型判断）
matched = df[df["match_success"]]
print(f"  匹配成功段: {len(matched)}")
print(f"  匹配距离中位: {matched['nearest_dist_m'].median():.1f}m")

print(f"\n参数推荐:")
print(f"  居住持续时间阈值: 4h（合理平衡点）")
print(f"  空间匹配阈值: 1000m（允许定位误差）")
print(f"  购物时间: 9-21点（商场营业时间）")
print(f"  旅游时间: 9-17点（景点开放时间）")
```

8.4 运行结果与解读

展示参数敏感性分析和边界情况统计。

8.5 课时小结

- 居住持续时间阈值4h是合理平衡点；
- 空间匹配阈值1km允许定位误差；
- 边界情况：持续=0、跨天段、匹配错误类型POI；
- 参数调优可根据实际数据调整。

三、本日小结

1. **出行目的总览**：居住、购物、旅游、其他四类，时间规则+空间匹配；
2. **驻留段识别**：连续驻留点合并，cumsum实现段ID；
3. **时间规则**：居住（>4h且夜间）、购物（9-21点）、旅游（9-17点）；
4. **静态数据**：GBK编码，读取时指定 encoding="gbk"；
5. **空间匹配**：驻留段中心与POI<1km取最近，向量化计算；
6. **purpose.py**：完整实现出行目的识别，输出 purpose_result.csv；
7. **结果分析**：其他最多，居住持续最长，购物/旅游在白天；
8. **参数调优**：4h居住阈值、1km匹配阈值是合理平衡点。

四、本日作业

1. 运行 purpose.py，记录最终目的分布和匹配率；
2. 运行可视化代码，查看4张图，写一段分析；
3. 思考：空间匹配是否应该修正时间规则的误判？为什么？
4. 预习 Day08：思考"如何从出行段中识别出行方式（地铁/公交/小汽车/其他）？"